

Проект «Эмулятор микрокалькулятора Электроника МК-61s mini»

https://github.com/UN7FGO/MK61S_MINI

Руководство по работе с терминалом

Версия инструкции: 06.09.2026

Терминал дает доступ к программной памяти, стеку и регистрам МК-61, внутреннему хранилищу, клавиатуре устройства, сервисным функциям и часам RTC. Настоящее руководство перенесено из Word-документа и приведено в соответствие с текущей прошивкой.

Подключение

Подключите МК61s mini к компьютеру по USB и откройте появившийся виртуальный последовательный порт в PuTTY, Tera Term, Arduino IDE Serial Monitor, screen, minicom или другой терминальной программе.

Рекомендуемые параметры:

- скорость 115200 бод;
- 8 бит данных, без контроля четности, 1 стоп-бит;
- без аппаратного и программного управления потоком;
- окончание строки CR, LF или CRLF;
- локальное эхо выключено: устройство само возвращает введенные символы;
- окно не меньше 80 столбцов и 40 строк;
- для специального символа индикатора полезна кодовая страница Windows-1251.

При USB CDC значение скорости фактически не задает скорость USB-передачи, но 115200 соответствует настройке прошивки и подходит для всех терминальных программ. В режиме USB-диск последовательный терминал остановлен. Чтобы снова работать с командами, выйдите из режима USB-диска клавишей ESC на устройстве.

После подключения прошивка печатает версию и приглашение:

```
МК61s mini ver. Jul 19 2026(12:34:56)
/>
```

Перед > показывается текущий каталог C5. После запуска это корень /; после cd /Programs приглашение станет /Programs>. Если полный путь не помещается во внутренний буфер приглашения, вместо него показывается . . . >; pwd по-прежнему позволяет запросить путь явно.

Введите he lp и завершите строку клавишей Enter. Команды и английские имена регистров вводятся ASCII-символами. Имя команды пишется в нижнем регистре; специальная запись регистра начинается с заглавной R.

Редактор строки и история

Терминал принимает CR, LF и пары CRLF/LFCR. В ANSI-совместимых клиентах, включая PuTTY и Tera Term, стрелки влево и вправо перемещают курсор, Backspace стирает символ перед ним, а Delete — символ под ним. Ввод в середине строки работает в режиме вставки; Home и End переходят к началу и концу строки. Дополнительно поддерживаются стандартные сочетания Ctrl+A, Ctrl+B, Ctrl+D, Ctrl+E и Ctrl+F.

Стрелки вверх и вниз листают историю. Команда history печатает сохраненные строки. При возврате из истории восстанавливаются как незавершенная строка, так и положение курсора в ней.

Хранятся до 8 последних неповторяющихся подряд команд общим объемом до 256 байт. Максимальная полезная длина одной строки — 239 символов. Переполненная строка целиком отбрасывается и не исполняется.

Как устроена память МК-61

Программируемые калькуляторы семейства МК-61 имеют отдельные память программы и память данных. Программная память состоит из байтовых кодов операций, а стек и регистры памяти хранят числа в собственном десятичном формате калькулятора. Поэтому команды для программы, стека и регистров разделены.

Для программной памяти доступны как шестнадцатеричные коды, так и ассемблерные мнемоники. Условные переходы оригинального калькулятора выполняют переход при невыполнении написанного на клавише условия. Из-за этого названия некоторых ассемблерных переходов выглядят зеркально по отношению к надписям на клавишах.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	1	2	3	4	5	6	7	8	9	,	/- /	ВП	Сх	В↑	Вх
1	+	-	×	÷	↔	F 10*	F e*	F lg	F ln	F arcsin	F arccos	F arctg	F sin	F cos	F tg	
2	F π	F √	F x²	F 1/x	F xʸ	F ○	K +	K -	K ×	K ÷	K ↔					
3	K 3	K 4	K 5	K 6	K 7	K 8	K 9	K ,	K /- /	K ВП	K Сх	K В↑				
4	п 0	п 1	п 2	п 3	п 4	п 5	п 6	п 7	п 8	п 9	п А	п В	п С	п Д	п ↑	
5	с/п	бп	в/о	пп	к ноп	к 1	к 2	F X≠0	F L2	F X≥0	F L3	F L1	F X<0	F L0	F X=0	
6	ип 0	ип 1	ип 2	ип 3	ип 4	ип 5	ип 6	ип 7	ип 8	ип 9	ип А	ип В	ип С	ип Д	ип ↑	
7	к X≠0	к X≠0 1	к X≠0 2	к X≠0 3	к X≠0 4	к X≠0 5	к X≠0 6	к X≠0 7	к X≠0 8	к X≠0 9	к X≠0 А	к X≠0 В	к X≠0 С	к X≠0 Д	к X≠0 ↑	
8	к бп 0	к бп 1	к бп 2	к бп 3	к бп 4	к бп 5	к бп 6	к бп 7	к бп 8	к бп 9	к бп А	к бп В	к бп С	к бп Д	к бп ↑	
9	к X≥0	к X≥0 1	к X≥0 2	к X≥0 3	к X≥0 4	к X≥0 5	к X≥0 6	к X≥0 7	к X≥0 8	к X≥0 9	к X≥0 А	к X≥0 В	к X≥0 С	к X≥0 Д	к X≥0 ↑	
A	к пп 0	к пп 1	к пп 2	к пп 3	к пп 4	к пп 5	к пп 6	к пп 7	к пп 8	к пп 9	к пп А	к пп В	к пп С	к пп Д	к пп ↑	
B	к п 0	к п 1	к п 2	к п 3	к п 4	к п 5	к п 6	к п 7	к п 8	к п 9	к п А	к п В	к п С	к п Д	к п ↑	
C	к X<0	к X<0 1	к X<0 2	к X<0 3	к X<0 4	к X<0 5	к X<0 6	к X<0 7	к X<0 8	к X<0 9	к X<0 А	к X<0 В	к X<0 С	к X<0 Д	к X<0 ↑	
D	к ип 0	к ип 1	к ип 2	к ип 3	к ип 4	к ип 5	к ип 6	к ип 7	к ип 8	к ип 9	к ип А	к ип В	к ип С	к ип Д	к ип ↑	
E	к X=0	к X=0 1	к X=0 2	к X=0 3	к X=0 4	к X=0 5	к X=0 6	к X=0 7	к X=0 8	к X=0 9	к X=0 А	к X=0 В	к X=0 С	к X=0 Д	к X=0 ↑	
F																

Классическая таблица кодов команд МК-61

По вертикали указана первая шестнадцатеричная цифра кода, по горизонтали — вторая.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	1	2	3	4	5	6	7	8	9	dot	neg	pow10	clr	push	preX
1	add	sub	mul	div	swap	e10	exp	lg	ln	asin	acos	atg	sin	cos	tg	?
2	pi	sqrt	sqr	rec	pow	rot	toM	?	?	?	toMS	?	?	?	?	?
3	inMS	mod	sgn	inM	int	frc	max	and	or	xor	not	rnd	?	?	?	?
4	sto0	sto1	sto2	sto3	sto4	sto5	sto6	sto7	sto8	sto9	stoA	stoB	stoC	stoD	stoE	?
5	hlt	jmp	ret	call	nop	?	?	jz	rpt2	jlz	rpt3	rpt1	jme	rpt0	jnz	?
6	ld0	ld1	ld2	ld3	ld4	ld5	ld6	ld7	ld8	ld9	ldA	ldB	ldC	ldD	ldE	?
7	jz[0]	jz[1]	jz[2]	jz[3]	jz[R4]	jz[5]	jz[6]	jz[7]	jz[8]	jz[9]	jz[A]	jz[B]	jz[C]	jz[D]	jz[E]	?
8	jmp[0]	jmp[1]	jmp[2]	jmp[3]	jmp[4]	jmp[5]	jmp[6]	jmp[7]	jmp[8]	jmp[9]	jmp[A]	jmp[B]	jmp[C]	jmp[D]	jmp[E]	?
9	jlz[0]	jlz[1]	jlz[2]	jlz[3]	jlz[4]	jlz[5]	jlz[6]	jlz[7]	jlz[8]	jlz[9]	jlz[A]	jlz[B]	jlz[C]	jlz[D]	jlz[E]	?
A	call[0]	call[1]	call[2]	call[3]	call[4]	call[5]	call[6]	call[7]	call[8]	call[9]	call[A]	call[B]	call[C]	call[D]	call[E]	?
B	sto[0]	sto[1]	sto[2]	sto[3]	sto[4]	sto[5]	sto[6]	sto[7]	sto[8]	sto[9]	sto[A]	sto[B]	sto[C]	sto[D]	sto[E]	?
C	jme[0]	jme[1]	jme[2]	jme[3]	jme[4]	jme[5]	jme[6]	jme[7]	jme[8]	jme[9]	jme[A]	jme[B]	jme[C]	jme[D]	jme[E]	?
D	ld[0]	ld[1]	ld[2]	ld[3]	ld[4]	ld[5]	ld[6]	ld[7]	ld[8]	ld[9]	ld[A]	ld[B]	ld[C]	ld[D]	ld[E]	?
E	jnz[0]	jnz[1]	jnz[2]	jnz[3]	jnz[R4]	jnz[5]	jnz[6]	jnz[7]	jnz[8]	jnz[9]	jnz[A]	jnz[B]	jnz[C]	jnz[D]	jnz[E]	?
F	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?

Таблица кодов с ассемблерными мнемониками

Клавиша калькулятора	Мнемоника	Смысл мнемоники
БП	JMP	безусловный переход, Jump
ПП	CALL	вызов подпрограммы, Call
В/О	RET	возврат, Return
С/П	HLT	останов, Halt
П→X	STO	запись, Store
X←П	LD	загрузка, Load
X≠0	JZ	переход при нуле

Клавиша калькулятора	Мнемоника	Смысл мнемоники
$X \geq 0$	JLZ	переход при отрицательном X
$X < 0$	JME	переход при X больше или равно нулю
$X = 0$	JNZ	переход при ненулевом X

Краткий справочник

Общие команды

Команда	Назначение
ver	Версия и дата сборки прошивки.
identity	Стабильная идентичность платы, сборки, USB и тома C5.
date	Чтение и установка даты и времени RTC.
alarm	Два аппаратных RTC-будильника A/B: разовый или ежедневный.
help	Актуальный список команд из самой прошивки.
history	История строк интерактивного терминала.

Программная память

Команда	Назначение
list	Вертикальная hex-таблица программной памяти.
dump	Последовательность hex-кодов программы.
pub	Листинг в журнальном формате.
lasm	Дизассемблированный листинг.
isa	Таблица ассемблерных мнемоник.
asm	Сборка мнемоник в программную память.
ins	Вставка одного opcode со сдвигом программы.
hin	Запись строки hex-байтов.
hout	Вывод программы в виде строк для hin.
clr	Очистка программной памяти с подтверждением.
set\$	Совместимая служебная форма hex-записи.

Состояние калькулятора и клавиатура

Команда	Назначение
reg	Регистры памяти R0...RE.
stk	Стек X, Y, Z, T, X1 и адрес исполнения.
poke	Запись числа в X, Y, Z или T.
1302	Регистр R микросхемы K145ИК1302.
ring	Активное кольцо памяти M.
R...=	Запись числа в R0...RE, а в расширенном режиме и RF.
kbd	Нажатие физической клавиши по scan-code.
cmd	Нажатие клавиш, соответствующих opcode МК-61.
reinit	Чистая инициализация калькулятора без перезагрузки устройства.
run	Запуск программы или открытие сохраненного файла.
disa	Переключение дизассемблера на дисплее.

Сценарии, индикация и звук

Команда	Назначение
open	Открытие или запуск файла из хранилища.
if	Выполнение следующей команды по условию.
print	Вывод строки, байтовых escapes и регистров на текущий экран; прежде всего для M61.
wait	Неблокирующая пауза M61-сценария в миллисекундах.
ret	Возврат из M61-сценария или обработчика ловушки.
bind	Привязка введенного opcode к обработчику M61.
led	Асинхронная последовательность состояний светодиода.
beep	Асинхронная последовательность звуков и пауз.

Хранилище

Команда	Назначение
save	Сохранение программы M61 в слот или по пути, с подтверждением.
load	Загрузка программы M61 из слота или по пути.
pwd	Полный путь текущего каталога.
cd	Смена текущего каталога.
ls, dir	Просмотр каталога или одной записи.
mkdir	Создание каталога; -p создаёт весь путь.
mv	Перемещение или переименование файла/каталога.
rm, del	Удаление файла; -r удаляет дерево.
rmdir	Удаление пустого каталога.
df	Физическая ёмкость, квота узлов и размер FAT12-кластера.
bench	Read-only замер чтения файла или последовательного диапазона C5.
fsget, fsput	Машинная передача файлов для tools/mkc.cmd.
smap	Карта числовых M61-слотов 0...99.
sdir	Каталог занятых числовых M61-слотов.
snm	Переименование числового M61-слота.
sdel	Удаление числового M61-слота с подтверждением.
sera	Форматирование всего хранилища с подтверждением.
fsls, fsm, fsstat	Синонимы ls, rm, df.
fsclean	Удаление записей нулевой длины.
vlog	Журнал трассировки виртуального FAT.

Сервис

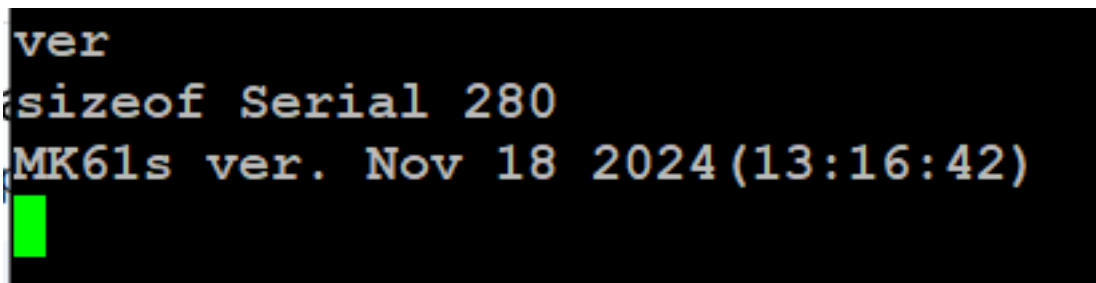
Команда	Назначение
prof	DWT-профиль времени ядра, дисплея, USB Screen, C5 и ZX0; снимок можно сохранить в текстовый файл.
crash	Просмотр, сохранение и очистка сохранённого аппаратного fault dump.
wdog	Состояние independent watchdog, причина последнего reset и retained breadcrumbs.
mpu	Состояние аппаратной защиты нулевого адреса, границы stack/heap и исполнения из SRAM.
display	Статус и стендовые тесты OLED1602/WS0010; в других профилях доступен только статус.
uscreen	Запуск ожидания desktop-клиента USB Screen, если возможность включена при сборке.
rst	Перезагрузка: подтверждение на устройстве либо явное <code>rst now</code> .
dfu	Немедленный переход в USB DFU-загрузчик.

Общие команды

ver — версия прошивки

```
ver
```

Команда печатает модель, дату и время сборки. Строка `sizeof Serial` — служебная диагностическая информация.



Пример вывода `ver`; версия на снимке может отличаться

identity — идентичность устройства и сборки

```
identity
```

Команда печатает одну версию машинно-читаемую строку: 64-битный public ID конкретного STM32, его короткий суффикс, фактическую USB serial строку STM32duino, отдельный FAT volume serial, build ID и профиль сборки. Полный заводской 96-битный UID наружу не выводится. Public ID стабилен после обновления прошивки и смены номера COM, но не является секретом или криптографическим ключом. `tools/mks.cmd` использует эту строку, чтобы не перепутать MK61s с другой STM32 и различать несколько калькуляторов.

```
MK61 ID v=1 public=0123456789ABCDEF short=89ABCDEF usb=112233445566 volume=A1B2C3D4  
build=12345678 profile=mini-v3-a00
```

date - дата и время

```
date
date ms
date 2026-07-19 14:35:00
```

Возможные ответы:

```
2026-07-19 14:35:07
2026-07-19 14:35:07.483
Set the date.
DATE OK
DATE ERROR: expected ms or YYYY-MM-DD HH:MM:SS
```

Без параметра команда печатает прежний seconds-only формат одной строкой; `date ms` добавляет согласованные доли секунды `.mmm`, не меняя совместимость старых хостовых программ. Если пользовательское время ещё не задавалось либо была потеряна резервная область RTC, следующей строкой выводится предупреждение `Set the date.` Параметр сразу задаёт новое значение: отдельное слово `set` не используется. Формат строгий, годы допустимы от 2000 до 2099, календарная дата проверяется с учётом високосных лет. Прошивка хранит введённое местное время и не пересчитывает часовой пояс. Полное описание приведено в документе `MK61s-mini-RTC.md`.

alarm — аппаратные будильники RTC

```
alarm
alarm a in 10
alarm b at 2026-08-30 07:15:00
alarm a daily 21:00:00
alarm b off
```

Без параметров выводятся retained-метаданные RTC и состояния двух независимых слотов A/B. `in` ставит точный относительно текущей субсекундной фазы разовый будильник через 1...31 536 000 секунд; `at` задаёт разовый календарный момент; `daily` повторяется ежедневно; `off` отменяет слот. Успех подтверждается `ALARM OK`, отказ проверки календаря, резервной записи или аппаратного включения — `ALARM ERROR`.

Срабатывание только подаёт короткий звук с учётом пользовательской громкости: оно не перехватывает экран и не запускает M61/APP. При нулевой громкости уведомление остаётся зарегистрированным, но беззвучно. Будильник способен разбудить питающийся STM32 из поддерживаемого режима сна, но не включает физически обесточенную плату. Команда запрещена из M61-сценариев и trap.

help — список команд

```
help
```

Список строится из того же реестра, по которому прошивка распознает команды, поэтому `help` — самый надежный способ проверить возможности установленной версии. В конце также выводятся специальные формы `R<r>=` и `set$`.

В F401 с `PortableApps=1` описания хранятся в `/System/HELP0.TXT` и `HELP1.TXT`. Сборщик извлекает их из ELF той же прошивки, не занимая её Flash. Оба файла начинаются с подписи набора команд; устанавливайте их вместе с `SETUP.APP` из комплекта. Если справка отсутствует, устарела или C5 занят, `help` печатает список имён команд и синтаксис восстановления файлов через `fsput`. Команды терминала остаются доступными. В обычной F411 сборке справка встроена.

history — история ввода

```
history
```

Команда нумерует строки от старой к новой. Стрелка вверх вызывает предыдущую строку, стрелка вниз возвращается к более новой или к строке, которую пользователь редактировал до входа в историю.

Программная память

list — вертикальная hex-таблица

```
list
```

Каждая ячейка выводится вместе с адресом шага программы.

```
list
00. 52 16. 43 32. 42 48. 10 64. 65 80. 11 96. 64
01. 6A 17. 87 33. 5B 49. 45 65. 0F 81. 5E 97. A8
02. 1D 18. 77 34. 37 50. 10 66. 01 82. 85 98. 65
03. 5C 19. 63 35. 6D 51. 31 67. 11 83. 6D 99. 0E
04. 18 20. 5E 36. 50 52. 6C 68. 11 84. 50 A0. 0E
05. 63 21. 27 37. 61 53. 11 69. 5E 85. 6B A1. 0E
06. 5E 22. 62 38. 69 54. 59 70. 78 86. 65 A2. 0E
07. 44 23. 43 39. 12 55. 64 71. 66 87. 11 A3. 0E
08. 62 24. 00 40. 4C 56. 65 72. 64 88. 5E A4. 0E
09. 0B 25. 42 41. 40 57. 0F 73. 11 89. 96
10. 43 26. 87 42. 64 58. 11 74. 59 90. 05
11. 00 27. 0B 43. 62 59. 59 75. 78 91. 64
12. 42 28. 42 44. 10 60. 63 76. 6B 92. 11
13. 87 29. 00 45. 44 61. 6D 77. 45 93. 5E
14. 42 30. 43 46. 65 62. 50 78. 65 94. 96
15. 00 31. 5D 47. 63 63. 27 79. 6E 95. 50
```

Пример вывода list

dump — непрерывный hex-листинг

dump

Удобен для быстрого копирования или сравнения последовательности opcode.

```
dump
52 6A 1D 5C 18 63 5E 44 62 0B 43 00 42 87 42 00
43 87 77 63 5E 27 62 43 00 42 87 0B 42 00 43 5D
42 5B 37 6D 50 61 69 12 4C 40 64 62 10 44 65 63
10 45 10 31 6C 11 59 64 65 0F 11 59 63 6D 50 27
65 0F 01 11 11 5E 78 66 64 11 59 78 6B 45 65 6E
11 5E 85 6D 50 6B 65 11 5E 96 05 64 11 5E 96 50
64 A8 65 0E 0E 0E 0E 0E 0E
```

Пример вывода dump

pub — журнальный формат

pub

Печатает программу колонками в формате, близком к публикациям программ для советских программируемых калькуляторов.


```

pub
00. B/O          30. X->ПЗ      60. 63          90. 5
01. П->ХА        31. FL0       61. П->ХД      91. П->Х4
02. Fcos         32. 42        62. C/П       92. -
03. Fx<0        33. FL1       63. ?         93. Fx=0
04. 18           34. 37        64. П->Х5     94. 96
05. П->Х3        35. П->ХД      65. Вх        95. C/П
06. Fx=0        36. C/П       66. 1         96. П->Х4
07. 44          37. П->Х1     67. -         97. РльПП8
08. П->Х2        38. П->Х9     68. -         98. П->Х5
09. /-/         39. *         69. Fx=0     99. В^
10. X->ПЗ       40. X->ПС      70. 78        A0. В^
11. 0           41. X->П0     71. П->Х6     A1. В^
12. X->П2       42. П->Х4     72. П->Х4     A2. В^
13. КВП7       43. П->Х2     73. -         A3. В^
14. X->П2       44. +         74. Fx>=0    A4. В^
15. 0           45. X->П4     75. 78
16. X->ПЗ       46. П->Х5     76. П->ХВ
17. КВП7       47. П->Х3     77. X->П5
18. Kx≠0 7     48. +         78. П->Х5
19. П->Х3       49. X->П5     79. П->ХЕ
20. Fx=0       50. +         80. -
21. 27         51. |x|      81. Fx=0
22. П->Х2       52. П->ХС     82. 85
23. X->ПЗ       53. -         83. П->ХД
24. 0           54. Fx>=0    84. C/П
25. X->П2       55. 64        85. П->ХВ
26. КВП7       56. П->Х5     86. П->Х5
27. /-/         57. Вх        87. -
28. X->П2       58. -         88. Fx=0
29. 0           59. Fx>=0    89. 96

```

Пример вывода pub

lasm — дизассемблированный листинг

```
lasm
```

Для каждого шага выводятся адрес, opcode и ассемблерная мнемоника. Второй байт двухбайтовой команды перехода показывается как ее аргумент.

```

iasm
0000 52 ret          0030 43 sto3          0060 63          0090 05 5
0001 6A ldA         0031 5D rpt0    42    0061 6D ldD          0091 64 ld4
0002 1D cos         0032 42          0062 50 hlt          0092 11 sub
0003 5C jme      18  0033 5B rpt1    37    0063 27 ?          0093 5E jnz      96
0004 18          0034 37          0064 65 ld5          0094 96
0005 63 ld3         0035 6D ldD          0065 0F preX        0095 50 hlt
0006 5E jnz      44  0036 50 hlt          0066 01 1          0096 64 ld4
0007 44          0037 61 ld1          0067 11 sub          0097 A8 call[8]
0008 62 ld2         0038 69 ld9          0068 11 sub          0098 65 ld5
0009 0B neg         0039 12 mul          0069 5E jnz      78    0099 0E push
0010 43 sto3        0040 4C stoC          0070 78          00A0 0E push
0011 00 0           0041 40 sto0          0071 66 ld6          00A1 0E push
0012 42 sto2        0042 64 ld4          0072 64 ld4          00A2 0E push
0013 87 jmp[7]      0043 62 ld2          0073 11 sub          00A3 0E push
0014 42 sto2        0044 10 add          0074 59 jl      78    00A4 0E push
0015 00 0           0045 44 sto4          0075 78          0076 6B ldB
0016 43 sto3        0046 65 ld5          0077 45 sto5        0078 65 ld5
0017 87 jmp[7]      0047 63 ld3          0079 6E ldE          0080 11 sub
0018 77 jz[7]       0048 10 add          0081 5E jnz      85    0082 85
0019 63 ld3         0049 45 sto5        0083 6D ldD          0084 50 hlt
0020 5E jnz      27  0050 10 add          0085 6B ldB          0086 65 ld5
0021 27          0051 31 mod          0087 11 sub          0088 5E jnz      96
0022 62 ld2         0052 6C ldC          0089 96
0023 43 sto3        0053 11 sub
0024 00 0           0054 59 jl      64    0090 05 5
0025 42 sto2        0055 64
0026 87 jmp[7]      0056 65 ld5
0027 0B neg         0057 0F preX
0028 42 sto2        0058 11 sub
0029 00 0           0059 59 jl      63

```

Пример вывода iasm

isa — мнемоники ассемблера

```
isa
```

Печатает поддерживаемые мнемоники в порядке opcode.

```
isa
00 0      01 1      02 2      03 3      04 4
05 5      06 6      07 7      08 8      09 9
0A dot    0B neg    0C pow10   0D clr    0E push
0F preX   10 add    11 sub     12 mul    13 div
14 swap   15 e10    16 exp     17 lg     18 ln
19 asin   1A acos   1B atg     1C sin    1D cos
1E tg     1F ?      20 pi     21 sqrt   22 sqr
23 rec    24 pow    25 rot     26 toM    27 ?
28 ?      29 ?      2A toMS   2B ?      2C ?
2D ?      2E ?      2F ?      30 inMS   31 mod
32 sgn    33 inM    34 int     35 frc    36 max
37 and    38 or     39 xor     3A not    3B rnd
3C ?      3D ?      3E ?      3F ?      40 sto0
41 stol   42 sto2   43 sto3   44 sto4   45 sto5
46 sto6   47 sto7   48 sto8   49 sto9   4A stoA
4B stoB   4C stoC   4D stoD   4E stoE   4F ?
50 hlt    51 jmp    52 ret     53 call   54 nop
```

Пример вывода isa

asm — ассемблерный ввод

```
asm [addr] <mnemonics>
asm 0000 1 2 add hlt
asm 3 4 mul
```

Необязательный addr — ровно четыре десятичные цифры начального адреса. Если адрес не указан, используется адрес трансляции, оставшийся после предыдущей успешной команды asm. Мнемоники разделяются пробелами; завершающий пробел не нужен.

Строка сначала разбирается целиком и только затем записывается. Неизвестная мнемоника или выход за границу памяти оставляют прежнюю программу без частичной записи.

```
asm 0005 sin cos div
Assembled to mk61 parsing address 5
from new TA (5)
Parsing: sin cos div
TA = 5 : 1C
Parsing: cos div
TA = 6 : 1D
Parsing: div
TA = 7 : 13
Parsing:
Unexpected token:
```

```
pub
00. 0
01. 0
02. 0
03. 0
04. 0
05. Fsin
06. Fcos
07. :
08. 0
09. 0
```

Пример ассемблерного ввода

ins — вставка opcode

```
ins <step> <opcode>
ins 10 20
```

step — десятичный номер шага, opcode — шестнадцатеричный байт 00...FF. Команда вставляет байт, сдвигает последующую программу и корректирует адреса переходов.

```

lasm
0000 34 int
0001 40 sto0
0002 01 1
0003 60 ld0
0004 12 mul
0005 5D rpt0 03
0006 03
0007 50 hlt
0008 00 0

ins 0004 50
lasm
0000 34 int
0001 40 sto0
0002 01 1
0003 60 ld0
0004 50 hlt
0005 12 mul
0006 5D rpt0 03
0007 03
0008 50 hlt
0009 00 0

```

Пример вставки команды

hin — ввод hex-байтов

```

hin <addr> <hex-bytes>
hin 0000 010203040506
hin 0024 0E01030000000006

```

Адрес должен содержать ровно четыре десятичные цифры. Байты записываются слитно полными парами hex-цифр. Пробелы внутри строки байтов не допускаются. Запись за 105-й классический шаг автоматически включает расширенную программную память, если сборка ее поддерживает.

```
hin 0000 526A1D5C18635E44620B430042874200438777635E276243
parsing address 0

52,6A,1D,5C,18,63,5E,44,62,B,43,0,42,87,42,0,43,87,77,63,5E,27,62,43,
Stream recived!
```

Пример ввода hin

hout — вывод строк для hin

```
hout
```

Вывод разбит на строки с четырехзначным начальным адресом и группами до 24 байтов. Эти строки можно сохранить в текстовый файл или передать другому пользователю и затем выполнить как команды hin.

```
hout
0000 526A1D5C18635E44620B430042874200438777635E276243
0024 0042870B4200435D425B376D506169124C40646210446563
0048 104510316C115964650F1159636D5027650F0111115E7866
0072 641159786B45656E115E856D506B65115E960564115E9650
0096 64A8650E0E0E0E0E0E9B
```

Пример вывода hout

clr — очистка программы

```
clr
Y
```

После clr терминал ожидает отдельную строку Y или y. Отмена выполняется отдельной строкой N или n. Любая другая команда отменяет устаревший запрос подтверждения и затем обрабатывается как обычная команда.

set\$ — совместимая форма записи

```
set$0000 01020304
```

Форма выполняет ту же проверяемую hex-запись, что и hin, но адрес примыкает к имени. Для новых команд и файлов M61 предпочтительнее читаемая форма hin.

Стек, регистры и внутренняя память

reg — регистры R0...RE

```
reg
```

Показывает все классические регистры памяти в формате индикатора МК-61.

```

reg
R0 = 1.0000000 02
R1 = 1.0000000 01
R2 = 0.0000000 00
R3 = 0.1000000 00
R4 = 0.0000000 00
R5 = 6.0000000 -01
R6 = -0.7000000 01
R7 = 0.0000037 07
R8 = 0.0000099 07
R9 = 1.0000000 01
RA = 1.0000000 02
RB = -4.0000000 01
RC = 1.0000000 02
RD = Г.ГГ0Г 00 00
RE = -3.9000000 01
IP: 99

```

Пример вывода reg

stk — стек

stk

Печатает X1, T, Z, Y, X и текущий адрес исполнения IP.

```

stk
X1 = 6.0000000 -01
T = 0.0000000 00
Z = 0.0000000 00
Y = 0.0000000 00
X = 0.0000000 00
IP = 99

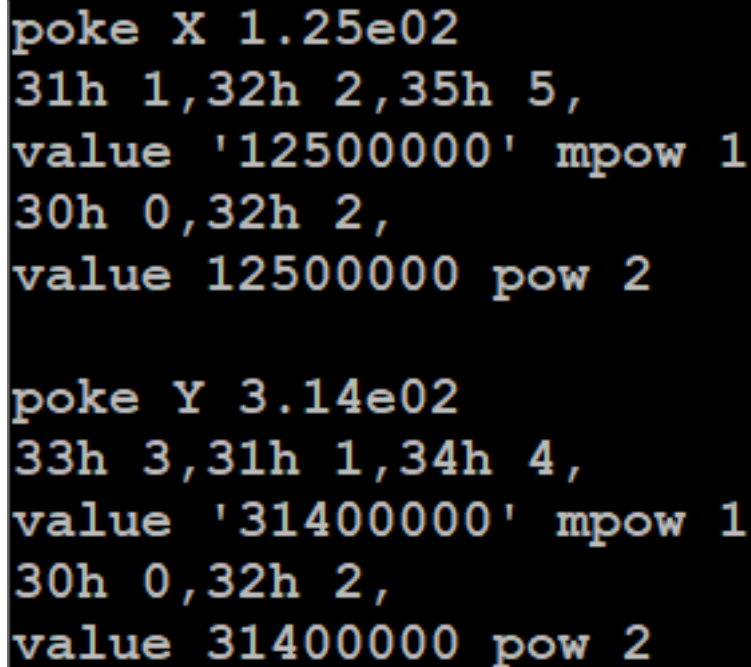
```

Пример вывода stk

poke — запись в стековый регистр

```
poke X 1.25e02  
poke Y -3.14
```

Допустимы X, Y, Z и T. Значение — конечное десятичное число, представимое форматом МК-61; порядок должен лежать в диапазоне от -99 до 99.



Пример записи poke

R<r>= — запись регистра памяти

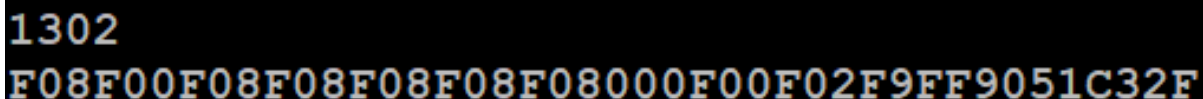
```
R0= 3.14  
RA= -1.25e02  
RE= 0  
RF= 42
```

Доступны R0...RE. RF разрешен только при включенной расширенной памяти. Значение проходит ту же проверку представимости, что и аргумент poke; произвольные внутренние BCD-тетрады этой формой записать нельзя.

1302 — регистр R K145ИК1302

```
1302
```

Это низкоуровневый диагностический вывод внутреннего регистра эмулируемой микросхемы.



Пример вывода 1302

ring — активное кольцо памяти M

```
ring
```


Печатает активные ячейки кольцевой памяти по эмулируемым микросхемам. Команда предназначена главным образом для диагностики ядра.

Клавиатура и запуск

kbd — физический scan-code

```
kbd <00..27>
kbd 02
```

Аргумент — шестнадцатеричный scan-code физической клавиатуры. Такая команда зависит от раскладки и нужна для точной имитации конкретной кнопки.

27 MENU ESC	22 ←	1D OK	18 →	13 [USER]	E P	9 ГРД	4 Г
F 26	$X<0$ 21 ШГ→	$L0$ 1C П→X	$\sin [x]$ 17 7	$\cos \{x\}$ 12 8	$\operatorname{tg}^{\max}$ D 9	$\sqrt{}$ 8 —	$1/x$ 3 ÷
25 ^K	$X=0$ 20 ←ШГ	$L1$ 1B X→П	$\sin^{-1} x $ 16 4	$\cos^{-1} 3H$ 11 5	$\operatorname{tg}^{-1} \vec{o'}$ C 6	$\pi \vec{o'}$ 7 +	X^2 2 ×
24 ↑↓	$X\geq 0$ 1F В/О	$L2$ 1A БП	e^x 15 1	$\lg$ 10 2	$\ln \vec{o''}$ B 3	$x^y \vec{o''}$ 6 ↔	$Bx \vec{C'}$ 1 е В↑
23 A↑	$X\neq 0$ 1E С/П	$L3$ 19 ПП	10^x НОП 14 0	↻ F a ·	Λ АВТ V b /-/	ПРГ ⊕ c ВП	CF ИНВ d Cx

Scan-code клавиатуры

При входе через kbd в блокирующее сервисное меню терминал не сможет выполнять следующие команды, пока пользователь не выйдет из меню на самом устройстве.

cmd — операция по opcode

```
cmd <00..EF>
cmd 53
```

Прошивка преобразует opcode МК-61 в соответствующую последовательность клавиш. В отличие от kbd, эта форма не зависит от физического scan-code и лучше подходит для сценариев. Например, cmd 53 соответствует операции ПП.

Старая команда exes больше не существует. Если нужен эквивалент нажатия ПП, используйте cmd 53.

reinit — чистое состояние калькулятора

```
reinit
```

Команда без подтверждения очищает программную память, стек, регистры, префиксы, ошибку, адрес исполнения и расширенное состояние МК-61. Текущий режим углов Р/Г/ГРД сохраняется. Активный M61-сценарий отменяется вместе с его ловушками trap и привязками bind.

Внутри обычного M61 команда продолжает тот же сценарий со следующей строки, но также снимает его активные trap и bind. В обработчике trap она запрещена. Не путайте её с clr, который очищает только программную память, и с rst, который перезагружает микроконтроллер и временно отключает USB.

Чтобы снять активные trap и bind, не очищая состояние калькулятора, удерживайте физическую клавишу Cx. Короткое нажатие Cx остаётся обычной операцией калькулятора.

run — запуск программы или файла

```
run
run DEMO
run DEMO.m61
```

Без аргумента команда имитирует последовательность F, / - /, B/O, C/П и запускает текущую программу. С именем она работает как open и открывает файл из хранилища.

Только внутри файла M61 допустима форма перехода на метку:

```
run :loop
```

В интерактивном терминале эта форма возвращает ошибку.

print — вывод на экран из M61

```
print "TEST"
print "X={X}, X1={X1}, X2={X2}, RF={RF}\r\n"
print "\x1B[3;5H*"
print "\x1B[2K\rstatus"
print "\xFF"
```

Команда пишет в текущий экраный буфер, не очищает его и не добавляет перевод строки. Обычный текст заменяет только свои ячейки. Она понимает \, \", \r, \n, \t, \b, \xHH, подстановки стека X/Y/Z/T/X1/X2 и памяти R0...RF (также короткие 0...F). RF требует расширенного режима. Суффикс :m выводит первую цифру нормализованной мантиссы, а :e — модуль десятичного порядка без ведущего нуля. Для X2 эти форматы неприменимы.

Разряды регистров выводятся через ту же таблицу 16 состояний, что и индикатор МК-61:

Тетрада	Знак на экране
0...9	0, 1...9 (0 — экраный знак нуля)
A	-
B	L
C	C
D	Г
E	E
F	пробел

Поэтому подстановка не теряет возникающие в разрядах L, C, Г, E, минус и пробел. {X}, {Y}, {Z}, {T}, {X1} и регистры памяти получают фиксированное регистровое представление. {X2} копирует непосредственно текущие 12 разрядов индикатора и положение запятой; именно её следует использовать для вывода снятого кадра программы МК-61.

Прошивка обрабатывает полезный монохромный поднабор ANSI: A/B/C/D/E/F, G, H/f, J/K, s/u и ESC 7/ESC 8; SGR не меняет монохромный экран. Координаты ограничены реальным размером активного дисплея. \xHH, включая \x00 и \xFF, передаёт один экраный код без перекодирования. В интерактивном терминале команда тоже доступна, но основной сценарий использования — .m61.

Первый успешный print из .m61 удерживает экран за сценарием до его завершения: промежуточная индикация работающего ядра подавляется, а выбранные кадры можно выводить из trap. Интерактивный print экран не захватывает.

wait <1..60000> доступна только внутри .m61 и асинхронно приостанавливает сценарий на указанное число миллисекунд. Внутри обработчика trap сохранённый контекст калькулятора остаётся приостановленным до ret.

ret и trap — возврат и перехват M61

ret допустим только в .m61: он возвращает из локальной метки, обработчика trap или bind, из вложенного сценария либо завершает корневой сценарий. Если к этому моменту в корневом сценарии активированы ловушки или привязки, прошивка оставляет наблюдатель включённым. ESC снимает его. Объявление

```
trap 105 run :message
```

останавливает программу перед opcode шага 105 и передаёт управление метке. Краткое описание языка находится в руководстве M61 (MK61s-mini-M61.md), а полный путь ловушки по коду, правила сохранения контекста и ограничения обработчика — в документе «Команда trap в M61: семантика и реализация» (MK61s-mini-M61-Trap.md).

Привязка

```
bind AE run :message
```

перехватывает введённый с клавиатуры opcode AE, поглощает исходную операцию и вызывает метку. Команды программы с тем же opcode не перехватываются. Обработчик bind обязан завершиться ret; одновременно допускается до восьми разных opcode. Полные правила приведены в руководстве M61.

disa — дизассемблер на дисплее

```
disa
```

Каждый вызов переключает режим верхней строки с дизассемблированной текущей операцией. Настройка действует не только в режиме ввода программы.

Пример последовательности действий

```
poke X 1.25e02
poke Y 3.14e02
kbd 02
stk
```

В X записывается 125, в Y — 314, затем нажимается клавиша умножения и проверяется результат в стеке. Конкретный scan-code следует сверять со схемой клавиатуры данной сборки.

```

poke Y 3.14e02
33h 3,31h 1,34h 4,
value '31400000' mpow 1
30h 0,32h 2,
value 31400000 pow 2
kbd 2
stk
X1 = 1.2500000 02
T = 0.0000000 00
Z = 0.0000000 00
Y = 0.0000000 00
X = 3.9250000 04
IP = 0

```

Пример последовательности команд терминала

Условия, светодиод и звук

if — условное выполнение

```

if <left><op><right> <command>
if x<0 open NEGATIVE.txt
if r0>0 run :loop
if y!=0 beep 1200,80

```

Операнды могут быть числами, стековыми регистрами x, y, z, t, x1 или регистрами памяти r0...re; rf доступен в расширенном режиме. Поддерживаются операции >, >=, <, <=, ==, !=. При ложном условии остаток строки не исполняется. При истинном условии он разбирается как обычная команда.

В интерактивном терминале полезны разовые условные действия. В M61 сочетание if и run :label образуют условные переходы и циклы. Внутри обработчика trap команда run :label является локальным вызовом: ret возвращает на следующую строку обработчика.

led — последовательность состояний светодиода

```

led 1
led 0
led 1,500,0,500,1

```

Нечетные элементы — состояния 0 или 1, четные — длительности в миллисекундах. Последнее состояние задается без длительности. Допустимо до 16 состояний, каждая пауза — до 65535 мс. Паттерн выполняется асинхронно.

beep — звуковой паттерн

```

beep 4000,100
beep 4000,100,0,50,2000,200

```

Аргументы идут парами частота Гц, длительность мс. Частота 0 задает паузу. Допустимо до 16 пар; каждое значение — от 0 до 65535. Воспроизведение асинхронное и использует текущую настройку громкости.

Файлы и хранилище

Файловая система C5 поддерживает каталоги и типы файлов M61, FOC, TBI, TXT, STATE.TXT, FMK, WBMP, CH8 и APP. Программы, тексты и шрифты ограничены 1536 байтами, изображения .wbmp - 1600 байтами, ROM .ch8 - 3584 байтами, исполняемые контейнеры .app - 20 544 байтами. Basename занимает до 31 байта корректного UTF-8, глубина дерева — до 32 каталогов. Количество объектов вычисляется из реально измеренной ёмкости flash: например, для 512 КиБ доступно 192 узла, для штатной W25Q128 на 16 МиБ — 4084. Обычный файл и каталог занимают по узлу; большой APP, ROM и расширение большого каталога дополнительно расходуют служебные узлы. Точные значения установленного устройства показывает df.

Пути могут быть абсолютными (/Programs/demo.m61) или относительными текущему каталогу (../Examples/demo.m61). Принимаются / и \, а также компоненты . и ... Весь аргумент с пробелами заключайте в одинарные или двойные кавычки:

```
cd "/My Programs"
mv 'old name.m61' '../Archive/new name.m61'
```

Расширения и сравнение имён не зависят от регистра в FAT-проекции, включая основные латинские, греческие и кириллические буквы. Символы FAT < > : " / \ | ? *, управляющие байты, ./, ., завершающие пробел/точка и зарезервированные DOS-имена не допускаются. Если расширение при чтении опущено, терминал принимает basename только при единственном совпадении; при нескольких типах нужно указать расширение.

Загружаемые APP на F401

Отдельных терминальных команд для APP нет. Это обычные файлы C5, поэтому для них работают ls, open, fsget, fsput, mv и rm. Пользовательский APP может иметь любое допустимое имя и находиться в любом каталоге:

Команда	Пример для .APP
ls /Apps	Показывает APP вместе с остальными файлами C5.
open /Apps/DEMO.APP	Проверяет, загружает в SRAM-overlay и запускает приложение.
fsget /Apps/DEMO.APP	Скачивает контейнер без преобразований.
fsput begin/data/end	Потоково устанавливает или заменяет контейнер после полной проверки.
mv /Apps/DEMO.APP /Archive/NEW.APP	Перемещает или переименовывает как обычный файл.
rm /Apps/DEMO.APP	Удаляет APP и делает его блоки доступными сборщику мусора.
df	Учитывает inode, служебные extent и физические блоки в общей C5.

Системные компоненты имеют тот же формат. FOCAL, TinyBASIC, просмотрщики WBMP и Markdown, консоль CHIP-8 и служебные экраны SETUP ищутся под каноническими путями /System/FOCAL.APP, /System/BASIC.APP, /System/WBMP.APP, /System/MARKDOWN.APP, /System/CHIP8.APP и /System/SETUP.APP. System — обычный видимый каталог: его можно создать и копировать целиком через USB или MKC. Фиксированного числа APP нет: предел задают свободное место, общая квота inode и ёмкость каталога. Host-тест создаёт 200 APP; это не предельное число.

open — открыть файл

```
open DEMO.m61
open /Manual/README.txt
open /Pictures/SCHEME.wbmp
open /Games/PONG.ch8
open "../FOCAL examples/TEST.foc"
open /Apps/DEMO.app
```

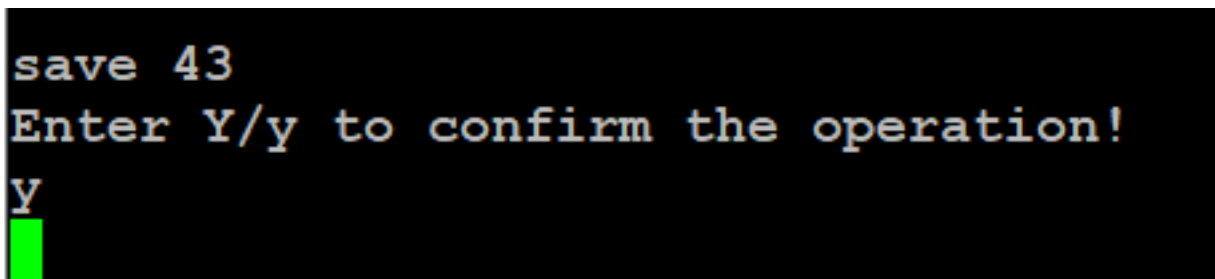
Файл открывается или запускается в соответствии с типом. В M61-сценарии вложенный файл выполняется с последующим возвратом на следующую строку родительского сценария.

Для .wbmp команда открывает тот же полноэкранный просмотрщик, что и Проводник; .ch8 запускает обработчик консольного типа C1. Оба формата требуют текущий графический экран. На LCD1602 они доступны только при активном USB-экране, иначе команда сообщает ГРАФИКА / НЕДОСТУПНА. Пользовательский .app загружается в общий 20-КиБ SRAM-overlay и получает команду APPLICATION_RUN. run <path> является синонимом open <path>; run без аргумента запускает текущую программу. Подробности правил M61 приведены в MK61s-mini-M61.md, а разработка и сборка исполняемых файлов — в [MK61s-mini-APP.md](#).

save — сохранить текущую программу

```
save 01
save /Programs/DEMO.m61
save "My Programs/DEMO 2"
Y
```

Принимается исторический номер 0...99 либо абсолютный/относительный путь M61 с необязательным .m61. Чисто цифровой аргумент всегда означает корневой совместимый слот, независимо от текущего каталога. Перед записью терминал ожидает отдельную строку Y/y; N/n отменяет операцию. Существующий M61 по тому же пути обновляется атомарно. Пустая программа не сохраняется.



Пример подтверждения save

load — загрузить M61

```
load 01
load ../Programs/DEMO.m61
```

В интерактивном терминале команда загружает программу в память. В M61-сценарии числовой слот или именованный M61 выполняется как вложенный сценарий и затем возвращает управление вызывающему файлу.

pwd и cd — текущий каталог

```
pwd
cd /Programs
cd ..
cd
```

pwd печатает абсолютный путь. cd <path> переходит в существующий каталог, а cd без аргумента возвращает в корень. Текущий путь отображается перед > в каждом приглашении терминала.

ls и dir — просмотр каталога

```
ls
ls /Programs
ls ../README.txt
dir
```

Без аргумента печатаются только непосредственные дети текущего каталога, без чтения и сортировки всего дерева. Для каталога выводятся строки d, для файла — f, размер в байтах и видимое имя с расширением. Аргументом может быть также один файл. APP перечисляются как обычные узлы C5. dir и fsls — синонимы ls.

mkdir — создать каталог

```
mkdir Programs
mkdir -p /Projects/FOCAL/Tests
```

Без -p родитель должен существовать. С -p недостающие компоненты создаются, а уже существующий путь считается успехом.

mv — переместить или переименовать

```
mv DEMO.m61 FINAL.m61
mv FINAL.m61 /Archive
mv /Projects/Old "/Projects/New name"
```

Если назначение — существующий каталог, исходное имя сохраняется. Иначе последний компонент задаёт новое имя. Тип файла изменять расширением нельзя, существующее назначение не перезаписывается, каталог нельзя переместить внутрь собственного поддерева или сделать дерево глубже 32 уровней. APP подчиняются обычным правилам.

rm и rmdir — удалить

```
rm DEMO.m61
rm -r /Projects/Obsolete
rmdir /Projects/Empty
```

rm удаляет файл немедленно. Для каталога требуется явный rm -r: дерево удаляется снизу вверх без рекурсивного расхода стека, после чего печатается число удалённых объектов. rmdir удаляет только пустой каталог. Корень удалить нельзя. Удаление APP атомарно убирает его из каталога; освободившиеся блоки позже возвращаются сборщиком мусора. del и fsrm — синонимы rm и также принимают -r.

df — ёмкость и квоты

```
df
```

Печатает измеренную физическую ёмкость SPI NOR, занятые/свободные/всего узлы, число видимых файлов и каталогов, служебные directory extents, виртуальный размер FAT12-кластера и объём сектора, зарезервированный под настройки. В конце без записи считываются JEDEC/SFDP/status внешней NOR и печатаются запрошенная частота SPI1, PCLK, аппаратный делитель и фактическая SCK. STM32 умеет выбирать только дискретные делители, поэтому запрос 20 МГц не гарантирует 20 МГц на выводе. В release-сборке следом выводится проверка целостности resident: state, обязательность, размер и смещение footer, совпадающие expected/build/actual, аппаратный или программный backend и число DWT-тактов. not-run означает, что метаданные footer не прошли проверку и CRC не запускался; это не следует принимать за успешную software-проверку. fsstat — синоним df. Строка POWER показывает аппаратный PVD, паспортные границы выбранной ступени, состояние write gate, события текущей загрузки, breadcrumb предыдущей загрузки и число отклонённых program/erase/MS-C-записей. state=low и state=recovering означают, что новые записи намеренно запрещены.

Строка ANALOG содержит время и число преобразований единого ADC-снимка, сырые коды VREFINT/temperature/VBAT, вычисленные VDDA, температуру кристалла MCU в десятых долях градуса и напряжение VBAT. bat=unknown/no-detector означает, что текущая плата не имеет отдельного детектора установленной батарейки: диагностическое напряжение плавающего входа нельзя считать доказательством наличия CR2032. В тесной production-сборке F401/UC1609 эта дублирующая диагностическая строка исключена ради обязательного запаса Flash; пользовательские показания в меню «Оборудование» остаются доступны.

bench — воспроизводимый замер чтения C5

```
bench file manual.md 5
bench flash 65536 3
bench random 1024 3
```

Первая форма читает существующий файл тем же API, которым пользуются просмотрщики, и подходит для сравнения времени открытия; размер ограничен 1536 байтами. Вторая читает без записи последовательный диапазон области данных C5 блоками по 512 байт. random тем же размером блока проходит псевдослучайные адреса с фиксированным seed, поэтому результаты разных прошивок сопоставимы. Для каждого прохода выводятся чистое время чтения и CRC-32, а затем min/avg/max и байт/с. Несовпавший CRC немедленно делает тест ошибочным. Команда ничего не создаёт, не стирает и не изменяет во внешней

Flash. Она входит в квалификационные F411-сборки. В тесном production-профиле F401/UC1609 лабораторная команда по умолчанию исключена ради проверяемого запаса внутренней Flash; её можно явно вернуть флагом MK61_ENABLE_READ_BENCHMARKS=1 для отдельного стендового образа.

fsget и fspu — машинная передача файлов

Эти команды являются транспортом двухпанельного менеджера tools/mk .cmd и обычно не вводятся вручную. Они передают произвольный поддерживаемый файл C5, не меняя текущую программу калькулятора:

```
fsget "/Programs/demo.m61"
fspu begin "/Programs/demo.m61" 128 3141592653
fspu data 0 00112233AABBCCDD
fspu end
fspu cancel
```

fsget отвечает строками @MKC:GET, @MKC:DATA и @MKC:END. Загрузка начинается fspu begin, принимает последовательные HEX-блоки не более 96 байт и завершается fspu end; ответы — @MKC:READY, @MKC:ACK и @MKC:DONE. Ошибка имеет вид @MKC:ERROR <code>. Для APP код PUT_FIRMWARE означает несовпадение resident-прошивки, а PUT_APP — неверный заголовок, размер, сжатый поток или CRC.

Размер и контрольная сумма проверяются до атомарной записи C5. Контрольная сумма совпадает с POSIX cksum; в неё входит длина файла. Если между блоками рабочую память занял другой режим, загрузка отклоняется как прерванная. Любая команда, кроме следующего fspu, отменяет незавершённый сеанс. fsget и fspu намеренно запрещены внутри M61-сценариев.

Имя назначения обязательно содержит поддерживаемое расширение. Допустимы .m61, .foc, .tbi, .txt, .state.txt, .fmk, .wbmp, .ch8, .app и старые псевдонимы .t1, .m2, .wbm; действуют обычные ограничения C5 по имени и размеру. CHIP-8 ROM размером от 1 до 3584 байт и APP размером от 64 до 20 544 байт можно передавать по любому пути.

При fspu входной APP не хранится целиком в RAM. Прошивка принимает транспортные блоки последовательно, собирает их в одном 512-байтовом рабочем буфере и сохраняет под отдельными staging-key. После проверки POSIX cksum контейнер перечитывается потоково. Проверка заголовка и совместимости, CRC сохранённых данных, распаковки и CRC готового образа временно использует блок нужного размера из общего 20-КиБ пула. Для ABI 4 выполняются ZX0, обратный BCJ при наличии флага и проверка/применение релокаций. Прежний ABI 3 использует фиксированный адрес; точный resident и payload NONE поддерживаются в ABI 2. Старый C5-файл в это время ещё не меняется. Только затем новые динамические блоки и дескриптор публикуются через COW/WAL. Ошибка оставляет прежний APP доступным; потеря питания даёт после перезапуска целиком старую либо целиком новую версию. Незавершённые staging-key удаляются при следующем сеансе или явном fspu cancel.

Совместимые числовые слоты M61

Команды ниже работают только с корневыми M61-файлами 0.m61...99.m61 и сохранены для привычного рабочего процесса.

smap — карта слотов 0...99

```
smap
```

Карта 10×10 показывает [X] для существующей программы M61 с числовым именем и [] для свободного номера.


```
smap
```

		0	1	2	3	4	5	6	7	8	9
0	-	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
1	-	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
2	-	[]	[]	[X]	[]	[]	[]	[]	[]	[]	[]
3	-	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
4	-	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
5	-	[]	[]	[]	[]	[]	[]	[X]	[]	[]	[]
6	-	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
7	-	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
8	-	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
9	-	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]

Пример карты smap

sdir — список числовых слотов

```
sdir
```

Печатает занятые числовые M61-слоты.

```
sdir
22. FISHING
56. BR%ED-2
```

Пример вывода sdir

snm — переименовать числовой слот

```
snm 01 BE-HAPPY
snm 22 FISHING
```

Номер лежит в диапазоне 0...99, новое имя содержит от 1 до 31 байта UTF-8. Переименование не выполняется, если исходного числового файла нет, имя недопустимо или видимое FAT-имя уже занято. Для перемещения в каталог используйте mv.

В ранней версии руководства в примерах ошибочно было написано sname. Зарегистрированное имя команды — snm.

sdel — удалить числовой слот

```
sdel 22
Y
```

Команда проверяет существование M61-файла с указанным числовым именем и ждёт отдельную строку Y/y. N/n отменяет удаление.

```
sdel 43

Clear slot #43
Enter Y/y to confirm the operation!
Y
```

Пример подтверждения sdel

sera — форматировать хранилище

```
sera
Y
```

Команда форматирует всю C5, а не только числовые M61-слоты. Будут удалены все файлы и каталоги; зарезервированный журнал настроек сохраняет свою отдельную область. Требуется отдельное подтверждение Y/y, отмена — N/n.

fsclean — удалить пустые записи

```
fsclean
```

Немедленно удаляет все файлы нулевой длины во всём дереве и печатает их количество. Полезно после оборванного или некорректного импорта через виртуальный USB-диск.

vlog — трассировка виртуального FAT

```
vlog
```

Выводит накопленный журнал USB/VFAT только в сборке с MK61_VFAT_TRACE. В обычной сборке сообщает, что трассировка пуста.

Сервисные команды

prof — аппаратный профиль времени

```
prof reset
prof start
prof stop
prof core 40
prof save PROFILE.TXT
prof
```

В поддерживаемой STM32-сборке команда использует счётчик тактов Cortex-M4 DWT CYCCNT. start включает сбор, stop останавливает её и печатает статистику, reset очищает все точки, а команда без аргумента печатает текущий снимок. Для каждой точки выводятся число измерений, минимум, среднее, максимум и сумма тактов. Частота ядра и измеренная цена самой пары чтений CYCCNT находятся в заголовке.

prof core [1..1000] запускает повторяемый тест ядра (по умолчанию ровно 40 полных core.step). Перед тестом прошивка сохраняет всё рабочее состояние калькулятора в уже существующий общий context-слот, временно создаёт известное чистое состояние ядра, а перед ответом восстанавливает и проверяет исходное; программа, стек и регистры пользователя не очищаются и не записываются во Flash. Команда отказывается работать во время RUN или когда context-слот занят M61/математическим backend.

Точки flash.read, flash.write, flash.verify и flash.erase отдельно показывают polling-чтение SPI NOR, page program, контрольное чтение после записи и стирание сектора. Поэтому end-to-end fsput может

дать выборки одновременно в `flash.write` и `flash.verify`.

Строки `SPI1` и `SPI1 DMA` показывают состояние владельца общей шины, сбалансированность захватов и освобождений, число DMA-передач, байтов и входов в `WFI`, а также `fallback`, ошибки и `timeout`. В профилях `mini` длинные буферы от 64 байт передаются через `DMA2 Stream 2/3 Channel 3`; короткие команды и адреса остаются на `polling`-пути. Физические профили `UC1609` пока целиком используют прежний `polling`-путь до отдельной проверки общего `SPI1` на реальном экране.

Строка `USB CDC RX` показывает защиту входной очереди `STM32 USB CDC`. `supported=1`, `linked=1` означает, что штатная сборка перехватила ошибочный путь `Arduino Core` с нулевым приёмным буфером: при заполнении очереди `OUT endpoint` отвечает `NAK` и возобновляется после чтения. `throttles` считает такие эпизоды `backpressure` и очищается командами `prof start` и `prof reset`.

Следующая строка `CLASSIC` относится к аппаратному регулятору скорости `RUN`: она показывает `backend` (`TIM9` на `STM32F401/F411`), фактический период, состояние таймера, число тиков и исполненных шагов, отброшенные тики, текущую очередь и её максимум. В нормальном прогоне `misted=0`, а `max_pending` не превышает 1; очередь аппаратно ограничена двумя шагами, чтобы после долгой занятости не возникал неограниченный догоняющий рывок. `prof start` и `prof reset` очищают также эти диагностические счётчики.

В конце снимка повторяются строки `POWER` и, если она включена для данного профиля, `ANALOG`. `ADC`-снимок выполняется только при печати диагностики и не работает фоном, поэтому не добавляет периодических `IRQ` и не сокращает `WFI idle`.

Строки `SLEEP` описывают `shallow Cortex-M WFI`: число проверок и фактических входов, суммарное/минимальное/среднее/максимальное время сна, последнюю маску блокировки и счётчики причин отказа. Первая консервативная политика разрешает сон только в верхнем `idle` остановленного калькулятора. Его запрещают `MSC`, непосредственно ожидающая обработка `terminal` или `USB Screen`, звук, клавиша или `debounce`, `CLASSIC`, отложенное действие, вложенный `viewer/menu` и отсутствие `SysTick wake`. Сам по себе открытый `CDC`-порт или подключённый `USB Screen` больше не удерживает `CPU` активным: входящие `USB`-пакеты будят его прерыванием, а `SysTick` не позднее следующего миллисекундного тика запускает обычный `scanner` всех строк клавиатуры. `STOP/STANDBY` и изменение `clock tree` не используются. `prof start` и `prof reset` очищают также эти счётчики.

`prof save [path.txt]` останавливает сбор и записывает тот же снимок как текстовый файл `C5`; путь по умолчанию — `PROFILE.TXT`. Запись собственного отчёта не попадает в сохранённые счётчики `flash.write`, поэтому файл остаётся непротиворечивым и его можно скачать обычной командой `fsget`.

Пока сбор не запущен, каждая точка выполняет только быструю проверку флага. Размер-чувствительная сборка может полностью удалить профилировщик через `-DMK61_ENABLE_DWT_PROFILER=0`; тогда команда отсутствует в `help`. Сам `WFI` независимо выключается через `-DMK61_ENABLE_IDLE_WFI=0`.

crash — сохранённый аппаратный fault dump

```
crash
crash show
crash save [path.txt]
crash clear
```

На `STM32F401/F411` обработчики `HardFault`, `MemManage`, `BusFault` и `UsageFault` записывают в 192-байтную область `.noinit` тип исключения, `pc/lr`, кадр регистров, `MSP/PSP`, маски `Cortex-M4`, `CFSR/HFSR`, адреса `fault`, `reset flags`, режим калькулятора и последние счётчики `CLASSIC`. Экспорт отчёта дополнительно содержит `public ID` платы и текущий профиль сборки, не меняя 192-байтный `retained ABI`. Запись защищена `CRC` и переживает программную перезагрузку; обработчик `fault` не обращается к `USB`, дисплею, `SPI` или файловой системе.

При следующем штатном запуске валидная запись автоматически сохраняется в `C5` как один из четырёх кольцевых файлов `CRASH0.txt...CRASH3.txt`. Если `C5` временно недоступна, `SRAM`-запись остаётся доступной. `crash` и `crash show` печатают полный отчёт, `crash save` записывает его по выбранному пути (по умолчанию `CRASH.TXT`), а `crash clear` явно удаляет только `retained SRAM-record`. Уже сохранённые файлы `C5` эта команда не удаляет. Когда записи нет, команда показывает `reset flags` текущей загрузки и результат проверки `.noinit layout`.

Преднамеренные `crash test ...` существуют только в специальной стендовой сборке с `MK61_ENABLE_FAULT_INJECTION=1`; в `release` эти действия и их код отсутствуют.

wdog — independent watchdog

```
wdog
wdog status
```

На STM32F401/F411 команда показывает аппаратный backend IWDG, состояние драйвера, номинальный timeout, число полностью завершённых foreground epoch и фактических reload, максимальный интервал между ними и отдельный признак IWDGRSTF. Watchdog запускается только в конце setup() и кормится в одной точке после обслуживания USB, terminal, звука, LED и дисплея; обработчики прерываний его не кормят.

Текущий номинальный timeout — 20 секунд (prescaler=256, reload=2499). Он оставляет запас на разброс частоты LSI и на две последовательные 5-секундные границы ожидания SPI NOR. После watchdog reset строка WDOG_previous показывает последний валидный .noinit breadcrumb и отличает обычную работу от преднамеренного стендового зависания. При этом ложный аппаратный fault dump не создаётся: причина reset и exception — разные диагностические каналы.

Терминальная команда dfu при уже работающем IWDG сначала оставляет короткий retained handoff и выполняет software reset. Следующая ранняя загрузка входит в системный ROM bootloader до повторного запуска watchdog, поэтому длительная DFU-прошивка не прерывается.

Действия wdog test starve и wdog test hang существуют только в специальной стендовой сборке с MK61_ENABLE_WATCHDOG_TEST=1; в release их код и подсказка отсутствуют.

mpu — аппаратные границы памяти

```
mpu
mpu status
```

На STM32F401/F411 MPU запрещает доступ к нулевому адресу и ставит 256-байтную недоступную область между статическими данными/heap и зарезервированным стеком. Команда показывает фактические границы RAM, _end, guard, текущий и минимальный наблюдавшийся MSP и запас над guard. Это наблюдаемый минимум с момента старта, а не полный анализ high-water для каждой функции.

На F411 вся SRAM дополнительно помечена execute-never. На F401 исполнение из SRAM сохраняется, потому что загружаемые System APP действительно запускаются там; защита нулевого адреса и stack/heap guard при этом остаются. Если linker разместит статические данные в guard или аппаратных MPU-регионов недостаточно, защита не включится и mpu явно покажет layout=invalid/enabled=0.

Преднамеренные mpu test guard|null|exec доступны только в стендовой сборке с MK61_ENABLE_MPU_TEST=1. В release код этих действий отсутствует; реальные MemManage-проверки выполнены вместе с сохранением crash dump.

display — OLED1602/WS0010

```
display
display status
display reinit
display on
display off
display test text
display test alphabet 0
display test symbols
display test map 0
display test ddram 0
display test row 0 17
display test cgram
display test zero
display test cursor
display test clear
display test home
display test entry
display test autoshift
display test sleep
display test graphics 0
display test restore
```

display status сообщает контроллер, FT, карту DDRAM, состояние BF (bf-seen, bf-timeouts, bf-fault), текущий режим character/graphics, владельца графической транзакции owner=none/api/qualification, состояние OLED, маршрут USB Screen, тайм-аут сна, фазу запуска и

число recovery. В профиле WEH001602A остальные подкоманды выполняют аппаратную приёмку: смешанный текст, алфавит, все 256 байтов CGROM, 2x64 DDRAM, независимую строку, CGRAM, cursor/blink, clear/home, entry shift и сон. Числа: alphabet 0..4, map 0..7, ddram 0..63, row 0..10..63.

display test zero скрыто записывает 256 нулевых байтов — существенно больше полевого сценария рассинхронизации — затем очищает DDRAM и должен показать ZERO/BF 256: OK / 4BIT SYNC: OK. В терминале ожидаются bf-seen=1 и bf-timeouts=0, bf-fault=0; bf-seen=0 означает, что DB7/RW или уровни надо исследовать логическим анализатором, даже если фиксированный нижний предел пока даёт вывод.

Тесты временно заменяют содержимое физического экрана. После них выполните display test restore. У графического qualification-pattern есть также аппаратный выход: первое полное нажатие любой клавиши восстанавливает character mode, CGRAM/DDRAM и прежний интерфейс, но само это нажатие интерфейсу не передаётся. Пока owner=qualification, обычные символьные записи намеренно блокируются, поэтому терминал и фоновая перерисовка не могут превратить GDRAM в мусор. Графика доступна только в отдельной сборке с MK61_WS0010_GRAPHICS_100X16=1 и не означает, что конкретный WEH001602A уже квалифицирован. Полный порядок приведён в MK61s-mini-WS0010.md.

uscreen — внешний USB-экран

uscreen

В сборке с MK61_ENABLE_USB_SCREEN=1 команда переводит firmware в ожидание handshake с приложением MK61 USB Screen. Desktop-клиент отправляет её автоматически сразу после открытия CDC-порта, поэтому выбирать пункт меню на устройстве не требуется. Обычный terminal обслуживается из общего idle-loop, поэтому команда принимается, когда foreground-интерфейс был открыт с клавиатуры устройства. Если блокирующий viewer или редактор запущен самой командой orep, текущий terminal-вызов ещё не завершён и следующая команда ждёт его возврата; в таком сценарии сначала выйдите из viewer физической клавишей. До успешного ATTACH физический экран остаётся включённым.

Команда не принимает аргументов и идемпотентна. В сборке с выключенным USB Screen она отсутствует в help. Полное описание handshake, приложения и выхода находится в MK61s-mini-USB-Screen.md.

rst — перезагрузка

rst
rst now

Подтверждение выполняется на дисплее и клавиатуре самого устройства. После подтверждения микроконтроллер перезагружается, а USB-порт временно исчезает. Форма rst now выполняет перезагрузку сразу и предназначена для автоматических аппаратных тестов; явное слово now защищает от случайного пустого Enter.

dfu — загрузчик DFU

dfu

Команда без дополнительного терминального подтверждения переводит устройство в режим USB Device Firmware Upgrade. Текущая терминальная сессия сразу заканчивается. Перед вводом убедитесь, что переход действительно нужен.

Команды в файлах M61

Строки сценария M61 используют тот же синтаксис, но выполняются с ограниченными правами. Разрешены:

- управление: run, open, load, if, ret, trap, bind;
- программа: hin, set\$, asm, ins, list, dump, pub, lasm, isa, hout;
- калькулятор: poke, R<r>=, reg, stk, 1302, ring, kbd, cmd, reinit;
- безопасные действия: ver, print, wait, led, beep.

Метки `:name`, `run :name`, `trap`, `bind` и `ret` существуют только внутри M61. Новая команда терминала автоматически не получает права сценария. В частности, `date`, `history`, `crash`, `wdog`, `uscreen`, команды удаления, форматирование, перезагрузка и DFU в M61 запрещены.

Полный синтаксис, вложенные файлы, метки и ограничения описаны в `MK61s-mini-M61.md`.

Подтверждения и осторожность

Терминальное подтверждение отдельной строкой `Y/N` используют:

- `clr`;
- `save`;
- `sdel`;
- `sera`.

Обычный `rst` подтверждается на самом устройстве, а явный `rst now` — нет. `rm/del/fsrm`, в том числе рекурсивный `-r`, а также `mv`, `rmdir`, `fsclean`, `snm` и `dfu` выполняются без терминального запроса. Для опасных команд проверяйте полный путь и тип файла до нажатия `Enter`.

Что изменилось относительно старой инструкции

- Команда часов `rtc [set ...]` заменена на `date [...]`; при неустановленном времени выводится предупреждение `Set the date.`
- Добавлены `help`, `history`, `ring`, `cmd`, `open`, `if`, `led`, `beep`, `bind`, `reinit`, именованные `save/load`, дерево C5 и команды `pwd`, `cd`, `ls`, `mkdir`, `mv`, `rm -r`, `rmdir`, `df`, `fsclean` и `vlog`.
- `run` теперь умеет открывать файл, а в M61 — переходить на метку.
- `snm` принимает имя длиной до 31 байта UTF-8; ошибочная форма `sname` удалена из примеров.
- Первый аргумент `ins` — десятичный номер шага, второй — `hex-opcode`.
- `asm` больше не требует завершающего пробела и не изменяет память при ошибке разбора.
- Устаревшая команда `exes` удалена; эквивалент клавиши ПП — `cmd 53`.
- Подтверждение можно отменить отдельной строкой `N` или `n`.

Для установленной прошивки окончательным источником списка команд остается команда `help`.